```
        # add a CHECK constraint
        execute <<-SQL
          ALTER TABLE distributors
            ADD CONSTRAINT zipchk
              CHECK (char_length(zipcode) = 5) NO INHERIT;
        SQL
      end
      dir.down do
        execute <<-SQL
          ALTER TABLE distributors
            DROP CONSTRAINT zipchk
        SQL
      end
    end

    add_column :users, :home_page_url, :string
    rename_column :users, :email, :email_address
  end
end
```

Using `reversible` will ensure that the instructions are executed in the right order too. If the previous example migration is reverted, the `down` block will be run after the `home_page_url` column is removed and right before the table `distributors` is dropped.

Sometimes your migration will do something which is just plain irreversible; for example, it might destroy some data. In such cases, you can raise `ActiveRecord::IrreversibleMigration` in your `down` block. If someone tries to revert your migration, an error message will be displayed saying that it can't be done.

3.10 Using the up/down Methods

You can also use the old style of migration using `up` and `down` methods instead of the `change`method. The `up` method should describe the transformation you'd like to make to your schema, and the `down` method of your migration should revert the transformations done by the `up` method. In other words, the database schema should be unchanged if you do an `up` followed by a `down`. For example, if you create a table in the `up` method, you should drop it in the `down` method. It is wise to reverse the transformations in precisely the reverse order they were made in the `up` method. The example in the `reversible` section is equivalent to:

```
class ExampleMigration < ActiveRecord::Migration
  def up
    create_table :distributors do |t|
      t.string :zipcode
    end

    # add a CHECK constraint
    execute <<-SQL
      ALTER TABLE distributors
        ADD CONSTRAINT zipchk
        CHECK (char_length(zipcode) = 5);
    SQL

    add_column :users, :home_page_url, :string
    rename_column :users, :email, :email_address
  end

  def down
    rename_column :users, :email_address, :email
    remove_column :users, :home_page_url

    execute <<-SQL
      ALTER TABLE distributors
        DROP CONSTRAINT zipchk
    SQL
```